STRINGFEST ANALYTICS

Financial Analytics with Python in Excel

HOSTED BY

plumsolutions

FM in EXCEL

Financial Modelling in Excel meetup

Forecasting, simulation, and analytics that go beyond formulas

A hands-on, 60-minute working session

Presented by George Mount · Stringfest Analytics

Thanks for having me

To Plum Solutions and the Financial Modelling in Excel meetup for hosting tonight.

This community lives at the intersection of Excel and rigorous financial modelling, exactly where Python in Excel earns its keep.

Tonight we push past what formulas do comfortably: forecasting, simulation, and analytics on real-sized financial data.





What you'll walk away with

- Summarize and analyze larger financial datasets without leaving Excel
- Build and evaluate data-driven forecasts (trend + seasonality)
- Run Monte Carlo simulations to model risk and uncertainty
- Test many assumptions and scenarios at once, and visualize the range
- Every line of code, the datasets, and a copy-paste cheat sheet to keep

STRINGFEST ANALYTICS

HOW TODAY WORKS

Open Excel and follow along with me

- You'll need Python in Excel enabled (Microsoft 365 on the Current Channel).
- Open the datasets workbook I shared, Revenue, Bookings, Simulation sheets.
- In a cell, type =PY(and Excel drops into Python. Ctrl+Enter runs it.
- When you see the red PY() CELL card, type the code and run it with me.
- Your numbers will match mine, we set a random seed for reproducibility.

PY() CELL

```
=PY(
    import pandas as pd
    import numpy as np
    df = xl("revenue[#All]",
            headers=True)
    df.head()
)
```

STRINGFEST ANALYTICS

THE MENTAL MODEL

Where formulas stop, Python starts

EXCEL FORMULAS DO WELL

- Single-point what-if (Data Tables)
- A handful of named scenarios
- Straight-line trend / FORECAST
- Fixed, well-understood inputs
- Live, linked model cells

PYTHON IN EXCEL UNLOCKS

- Trend + seasonality forecasting
- Thousands of Monte Carlo trials
- Full distributions, not one number
- Decomposition & diagnostics
- Repeatable, seeded, auditable code

PART 1 OF 2

Forecasting revenue

Resample · smooth · decompose · Holt-Winters · evaluate

STRINGFEST ANALYTICS

STEP 1, UNDERSTAND THE SERIES

Load revenue and index it by time

- xl() pulls the Revenue table straight into a pandas DataFrame.
- Convert the date column so Python treats it as real time, not text.
- Set date as the index, now you can resample, shift, and compare by time.
- This monthly SaaS revenue rises over six years with a clear yearly rhythm.

PY() CELL

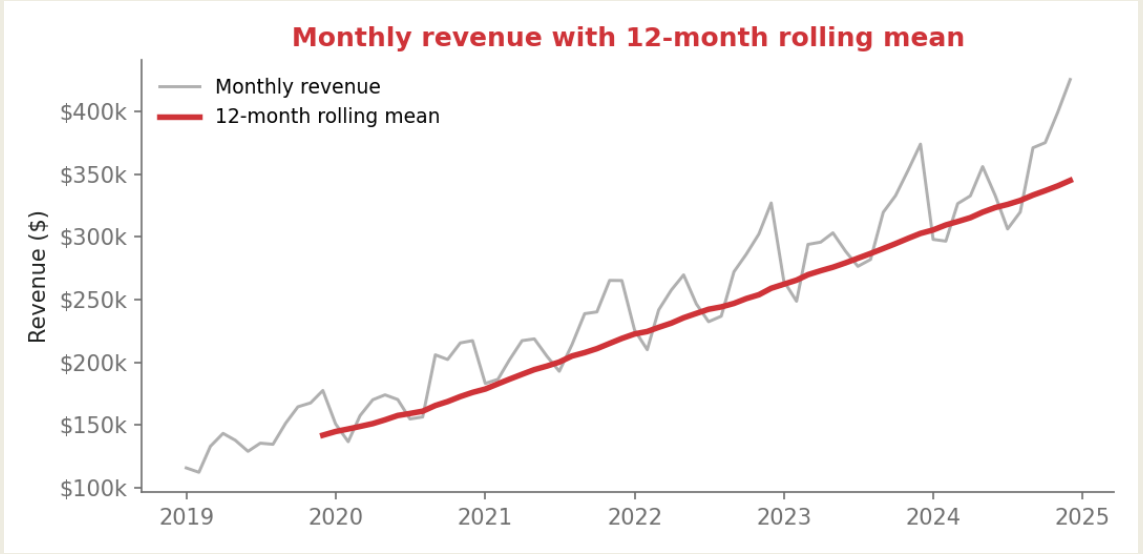
```
df = xl("revenue[#All]",
        headers=True)
df['date'] = pd.to_datetime(
    df['date'])
df = df.set_index('date')
df.head()
```

STRINGFEST ANALYTICS

STEP 2, REDUCE THE NOISE

Resample and smooth to see the trend

- resample('QE').mean() rolls months into quarters.
- rolling(window=12).mean() is the moving average you know, over a full year it cancels the seasonal swing.
- The red line makes the underlying growth unmistakable.



PY() CELL

```
quarterly = df.resample('QE').mean()
df['rolling'] = df['revenue'].rolling(window=12).mean()
```

STRINGFEST ANALYTICS

STEP 3, SEPARATE THE PIECES

Decompose: trend + seasonal + residual

- Observed = Trend + Seasonal + Residual.
- The trend is the smooth long-term climb; the seasonal wave repeats every 12 months.
- A near-random residual means trend + seasonality explain most of the variation.
- The verdict: both trend AND seasonality are present, so the method must handle both.

PY() CELL

```
from statsmodels.tsa.seasonal \
  import seasonal_decompose

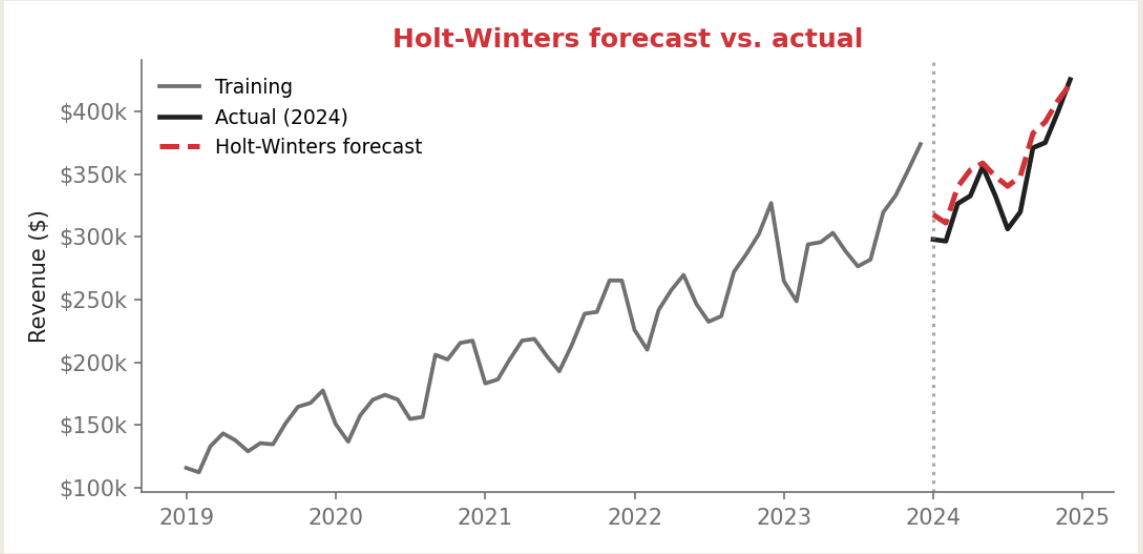
result = seasonal_decompose(
  df['revenue'],
  model='additive',
  period=12)
result.plot()
```

STRINGFEST ANALYTICS

STEP 4, FORECAST

Holt-Winters: trend and seasonality together

- Holt-Winters extends exponential smoothing to model trend AND seasonality.
- The forecast doesn't run in a straight line, it keeps rising AND keeps the yearly rhythm.
- A plain trendline would flatten those seasonal peaks and troughs.



PY() CELL

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
fit = ExponentialSmoothing(df['revenue'], trend='add', seasonal='add', seasonal_periods=12).fit()
forecast = fit.forecast(12)
```

STRINGFEST ANALYTICS

STEP 5, PROVE IT

Evaluate against a naive baseline

- Split in time: train through 2023, hold out 2024. Never shuffle time series.
- Mean Absolute Error (MAE) = average miss, in dollars, easy to defend.
- Baseline = 'next month equals last month.' Always beat something.
- Here Holt-Winters MAE is roughly a third of the naive baseline's.

PY() CELL

```
train = df[df.index < '2024-01-01']
test  = df[df.index >= '2024-01-01']
fit = ExponentialSmoothing(
  train['revenue'], trend='add',
  seasonal='add',
  seasonal_periods=12).fit()
fc = fit.forecast(len(test))
mae = (fc - test['revenue']).abs().mean()
```

STRINGFEST ANALYTICS

THE JUDGMENT CALL

Match the method to the data

- **Revenue: strong trend + stable seasonality -> Holt-Winters.**
- **Bookings: trend but NO reliable yearly pattern -> Holt's method (trend only).**
- Force seasonality onto the bookings series and the model invents peaks that aren't there.
- The decomposition plot tells you which camp you're in before you fit anything.
- Bigger toolbox beats a bigger hammer. Trend + seasonality -> Holt-Winters. Trend only -> Holt. Neither -> simple smoothing.

PART 2 OF 2

Simulating risk

From single-point estimates to full distributions of outcomes

STRINGFEST ANALYTICS

THE PROBLEM

One number hides the risk

- Stock 1,100 units at $30, sell at $50. Expected demand 1,000 -> 'profit $20,000.'
- But demand is uncertain. Sell 800 and profit falls to $7,000.
- Sell 1,300 and you cap out at inventory, the upside is limited, the downside isn't.
- **A single forecast tells you the middle. It never tells you how often the bad case shows up.**

$20,000

the single-point answer

?

how wide is the range,
and how often do we lose?

STRINGFEST ANALYTICS

STEP 1, MODEL THE UNCERTAINTY

Generate thousands of demand draws

- Pick a distribution that encodes your assumption, here, normal demand.
- NumPy draws 10,000 demand values centered on 1,000, s.d. 200.
- np.maximum(..., 0) keeps demand realistic (never negative).
- One seed makes the whole simulation reproducible and auditable.

PY() CELL

```
np.random.seed(42)
N = 10000
demand = np.maximum(
  np.random.normal(
    1000, 200, N),
  0)
```

STRINGFEST ANALYTICS

STEP 2, APPLY THE BUSINESS LOGIC

Compute profit for every scenario

- Units sold = MIN(demand, inventory), you can't sell what you don't stock.
- That MIN is the whole story: it caps the upside but not the downside.
- Revenue minus the cost of ALL inventory (sold or not) gives profit.
- Vectorized: 10,000 scenarios calculated in one shot, no drag-fill.

PY() CELL

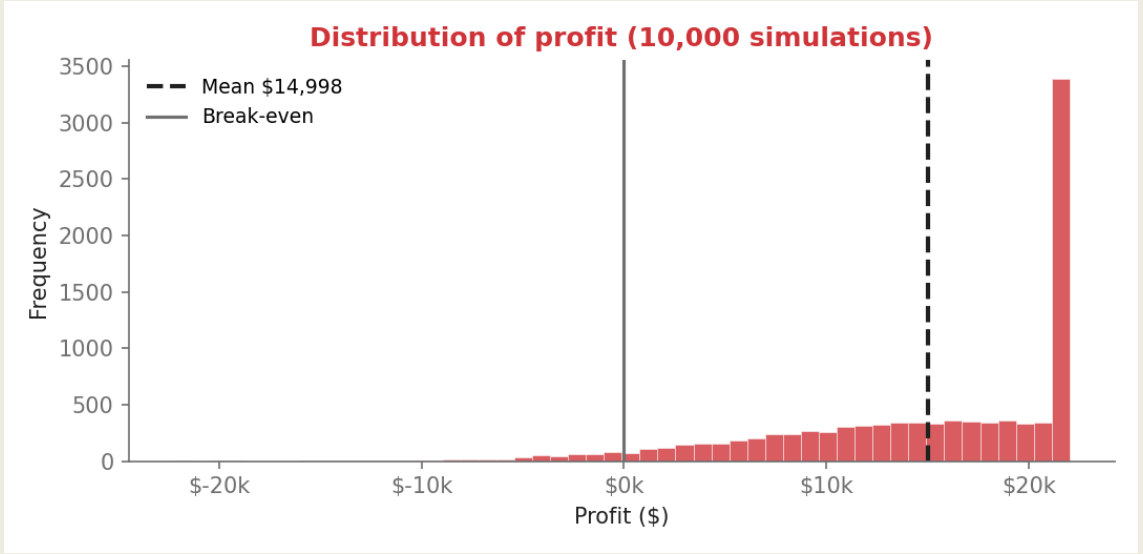
```
price, cost, inv = 50, 30, 1100
units = np.minimum(
  demand, inv)
revenue = units * price
profit = revenue - inv*cost
```

STRINGFEST ANALYTICS

STEP 3, READ THE DISTRIBUTION

Summarize with percentiles, not just the mean

- Mean and median: the typical outcome.
- 10th / 90th percentiles: a concrete 'reasonable range' you can plan against.
- Probability of loss answers the question the mean can't: how often do we lose?



PY() CELL

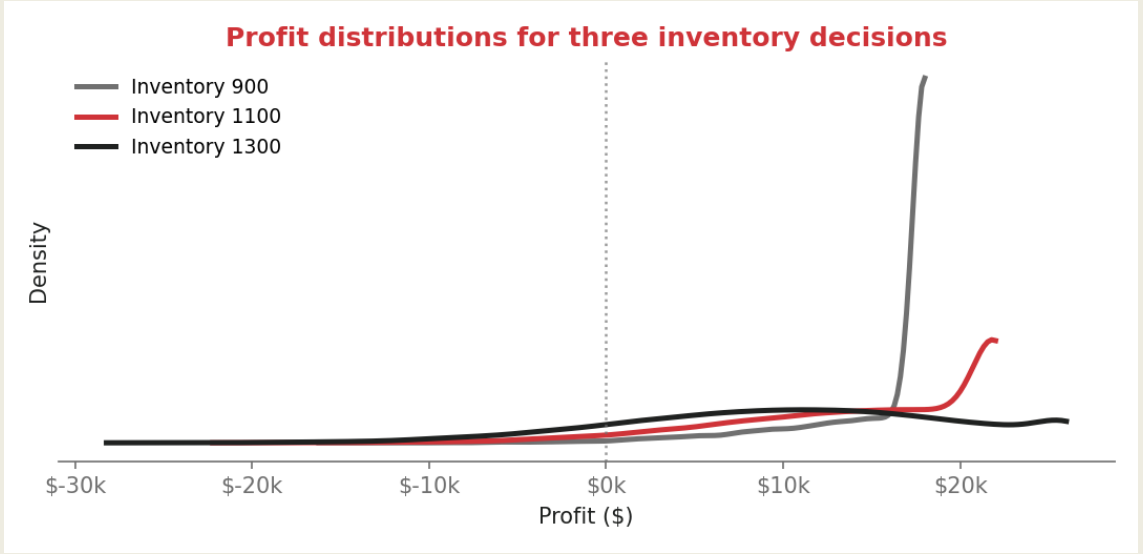
```
p10, p90 = np.percentile(profit, [10, 90])
prob_loss = (profit < 0).mean()
summary = pd.DataFrame({'mean':[profit.mean()], 'p10':[p10], 'p90':[p90], 'p_loss':[prob_loss]})
```

STRINGFEST ANALYTICS

STEP 4, THE MAIN EXERCISE

Compare decisions, not just averages

- Hold demand fixed; vary the decision, inventory of 900, 1,100, 1,300.
- Conservative (900): tight distribution, limited downside.
- Aggressive (1,300): more upside, but the tail crosses into losses.
- **Now it's a risk-tolerance choice, not a single ranking.**



PY() CELL

```
for inv in [900, 1100, 1300]:
    p = np.minimum(demand, inv) * price - inv * cost
    print(inv, round(p.mean()), round(np.percentile(p,10)), (p<0).mean())
```

STRINGFEST ANALYTICS

HOW THIS CONNECTS TO EXCEL

You already do this, Python scales it

Scenario Manager -> a few named cases. Data Tables -> one or two inputs on a grid.

Monte Carlo -> you define distributions and let the model explore thousands of combinations automatically.

Python adds a seed for reproducibility, richer distributions (skew, rare events), and full distributions instead of a grid of points.

Same instincts you already have, just past the point where the grid gets unwieldy.

STRINGFEST ANALYTICS

RECAP

Two new tools in your Excel toolkit

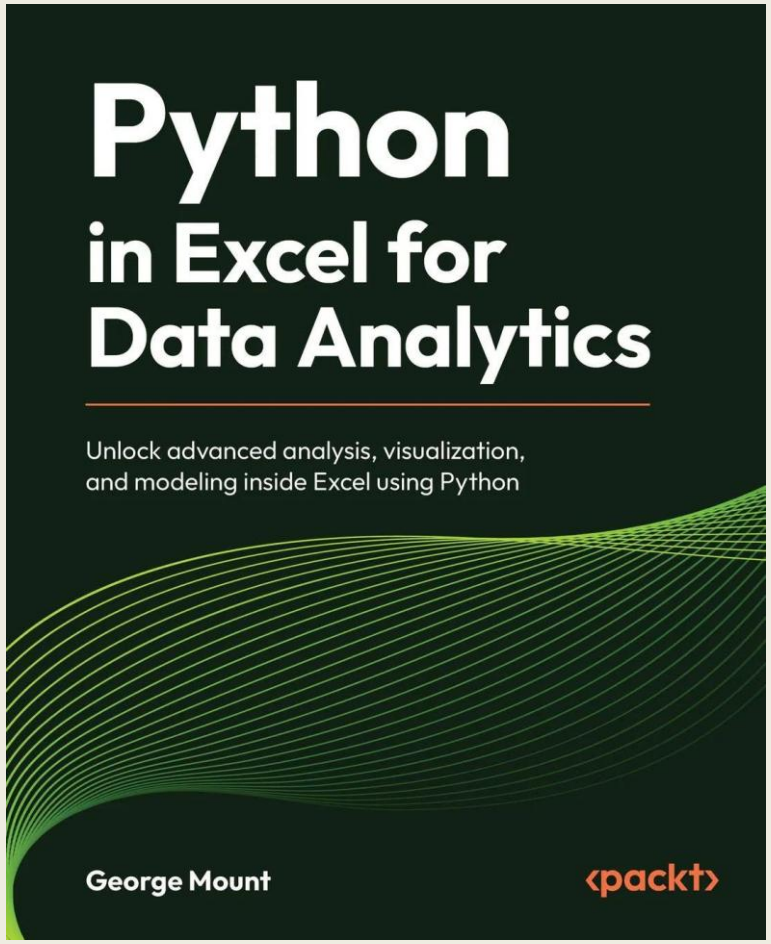
FORECASTING

- Index by time, then smooth
- Decompose to see structure
- Holt-Winters for trend + season
- Split by time; beat a baseline
- Match the method to the data

SIMULATION

- Sample from a distribution
- Apply the business logic
- Summarize with percentiles
- Read probability of loss
- Compare decisions by risk

STRINGFEST ANALYTICS



Want the whole toolkit?

Everything tonight, and much more, is in my forthcoming book:

Python in Excel for Data Analytics

Unlock advanced analysis, visualization, and modeling inside Excel using Python (Packt).

Forecasting, Monte Carlo simulation, regression, text analytics, and more, all inside Excel.

Rolling out by the end of the year.

Get launch-day notice: stringfestanalytics.com/book

STRINGFEST ANALYTICS

BEFORE YOU GO

Your take-home kit + one action

You're leaving with:

- The datasets workbook (Revenue · Bookings · Simulation)
- The prompt/code cheat sheet, every line from tonight
- The follow-along exercises with solutions

This week: open one of your own monthly series and run a Holt-Winters forecast, then beat the naive baseline.

STRINGFEST ANALYTICS

Thank you

George Mount · Stringfest Analytics

george@stringfestanalytics.com · stringfestanalytics.com

Thanks to Plum Solutions & the Financial Modelling in Excel meetup for hosting.

Book launching end of year -> stringfestanalytics.com/book

